C++ Programming Problem Set

Module: Static Members (Class-Level Data, Functions & Storage Duration)

CS-SIMPLIFIED

Author: Gaurav Kandpal

Target Concept: Static Members & Class Scope Mechanics

Integrates: Encapsulation, References, Arrays, Dynamic Memory & OOP

Instructions & Pedagogical Guidelines

This problem set focuses on class-level static mechanics in C++. You will construct programs testing persistent static local variables, mandatory file-scope external definitions of static data members, static member functions operating without instance contexts (no `this` pointer), object instantiation counters, and global configuration state management across dynamic instance arrays.

Level 1: Static Local Lifetime, Member Variables & Mandatory Definition

Problem 1: Persistent Function Execution Counter & Storage Duration

Static Local Variable | Storage Duration | Automatic Inits

Write a function `void trackInvocation()` containing an automatic variable `int stackCount = 0;` and a static local variable `static int staticCount;`. Call `trackInvocation()` 5 times inside a loop in `main()`. Output both counts per iteration to demonstrate that static local variables are automatically initialized to zero, allocated once in the Data/BSS segment, and persist across function calls.

Static Storage Duration | Data/BSS Segment | Zero Initialization

Iteration Step	Observed Memory State
`Call 1 to 5 inside main() loop`	`Iter 1: stack=1, static=1 | Iter 5: stack=1, static=5 Static variable retained value across stack frame pops!`

Problem 2: Public Static Data Member & Mandatory External Definition

Static Member Variables | Scope Resolution (::) | Linker Rules

Define a class `Configuration` with an instance member `int localSetting` and a public static member variable `static float globalTaxRate;`. In `main()`, demonstrate accessing `Configuration::globalTaxRate` before instantiating any objects. Show that omitting the mandatory file-scope external definition `float Configuration::globalTaxRate = 18.0f;` causes a fatal linker error (undefined reference).

Class-Level Scope (::) | Mandatory File-Scope Definition | Linker Diagnostics

Access Variant	Compiler & Linker Output
`Configuration::globalTaxRate = 18.0f;`	`Valid (Accesses single shared storage in Data Segment)`
`Omitted float Configuration::globalTaxRate;`	`LINKER ERROR: Undefined reference to Configuration::globalTaxRate`

Problem 3: Live Instance Counter & Shared Class Memory

Shared State | Object Counting | Constructors

Create a class `UserSession` with private `int userID` and private `static int activeUserCount`. Increment `activeUserCount` when a session initializes and add a public static member function `static int getActiveCount()`. Instantiate 3 instances on the Stack. Print `getActiveCount()` via the class name to verify that static variables are shared across all instances without duplicating space in individual objects.

Shared State Tracking | Object Footprint Independence | Public Static Accessor

Call Site Sequence	Output Summary
`UserSession u1, u2, u3;`	`Active Session Count: 3 (Shared single variable across all 3 objects)`

Level 2: Static Functions, Compiler Rules & Arrays of Objects

Problem 4: Static Function Constraints (No 'this' Pointer & Instance Gating)

Static Member Functions | 'this' Pointer | Instance Member Access

Demonstrate the architectural boundaries of static member functions. Define a class `Account` with private instance variable `double balance` and private static member `static float interestRate`. Implement a static member function `static void updateRate(float newRate)`. Attempting to write `balance = 500.0;` inside `updateRate()` causes a compiler error. Document why static functions lack an implicit `this` pointer.

```
// Static Member Function Constraint: static void updateRate(float newRate) {
interestRate = newRate; // OK: Accessing static data member! // balance = 500.0; //
ERROR: Invalid use of member 'balance' in static function! }
```

Operation inside Static Method	Compiler Evaluation
`interestRate = newRate;`	`SUCCESS: Modifies shared class variable in Data Segment.`
`this->balance = 1000.0;`	`ERROR: 'this' is unavailable for static member functions!`

Problem 5: Bank-Wide ROI Synchronizer across Account Arrays

Array of Objects | Shared Static State | Pass-by-Reference

Design a class `SavingsAccount` with instance `balance` and static `rateOfInterest`. Implement methods `void setBalance(double b)`, `void display()`, and `static void setROI(float r)`. Populate an array of 4 accounts with distinct balances in `main()`. Call `SavingsAccount::setROI(6.5f)` and output all accounts to prove that updating a static rate via the class name immediately alters interest calculations for all array elements.

Synchronized State | Array Iteration | Class-Wide Updates

Action Performed	Array Output Report
`Execute SavingsAccount::setROI(6.5f)`	`Acc #1 ($1,000): Interest $65.00` `Acc #4 ($5,000): Interest $325.00 (Updated across all objects!)`

Problem 6: String Sanitizer & Global Token Counter

std::string | Static Accumulator | Character Filters

Create a class `TextProcessor` containing a public static utility function `static std::string sanitizeAndCount(const std::string &rawText)` and a private static member `static int totalSymbolsFiltered`. The static function replaces non-alphanumeric characters with underscores, updates `totalSymbolsFiltered` by reference, and returns the cleaned string. Process 3 sentences in `main()` and display the total filter count via `TextProcessor::getFilterCount()`.

Static String Utility | Global Character Filtering | Const Reference Input

Sentences Processed	Global Static Tracker Output
`"Hello@World!", "C++2026#"`	`Sanitized: "Hello_World_", "C__2026_"` `Total Global Symbols Filtered: 4`

Level 3: Dynamic Heap Allocation, Singleton Pattern & Capstone

Problem 7: Dynamic Heap Object Allocation & Static Lifetime Protection

Dynamic Memory | new / delete | Heap Object Tracker

Combine dynamic Heap allocation with static tracking. Define a class `DynamicNode` with `static int activeHeapNodes` and `static int totalAllocations`. Increment both in a helper method called after `new DynamicNode()`. Allocate an array of 5 dynamic pointers in `main()`, delete 2 of them using `delete ptr`, and verify that `activeHeapNodes` accurately reflects live heap objects while `totalAllocations` preserves historical counts.

```
// Tracking Dynamic Heap Allocations via Static Members: DynamicNode* node1 = new DynamicNode(); // activeNodes: 1, totalAlloc: 1 DynamicNode* node2 = new DynamicNode(); // activeNodes: 2, totalAlloc: 2 delete node1; // activeNodes: 1, totalAlloc: 2
```

Heap Operations	Static Memory Tracker Output
`Allocated 5 objects on Heap; deleted 2`	`Live Heap Objects: 3 | Total Historical Allocations: 5`

Problem 8: The Static Singleton System Instance Pattern

Design Patterns | Private Constructor | Static Self-Pointer

Implement a fundamental OOP Singleton pattern. Define a class `DatabaseConnection` with a private constructor (preventing arbitrary new or stack instantiations). Declare a private static self-pointer `static DatabaseConnection* instancePtr;`. Create a public static member function `static DatabaseConnection* getInstance()` that allocates the single object instance on demand if `instancePtr == nullptr`.

Singleton Pattern | Private Constructor | Static Self-Pointer

Call Site	Instance Address Verification
`db1 = getInstance(); db2 = getInstance();`	`db1 Addr == db2 Addr (0x7ffe80) -> Single shared system instance!`

Problem 9: Cumulative Capstone: Industrial IoT Sensor Data Aggregator

Comprehensive Integration | Static Engine | Dynamic Arrays

Build a centralized IoT Industrial Monitoring Engine. Define a class `SensorNode` containing private `sensorID` (int), `lastReading` (double), static `totalTelemetryEvents` (int), static `globalMaxTemp` (double), and static `systemStatus` (std::string). Implement static member functions:
1. `static void logTelemetry(int id, double temp)` — Updates reading, increments total events, and checks global max.
2. `static void printSystemSummary()` — Outputs global telemetry metrics without requiring instance objects.
Simulate processing 6 sensor readings from multiple units in `main()` and display the system report.

Static IoT Telemetry Engine | Global Max Tracking | Class-Wide Analytics | Complete Module Capstone

Telemetry Stream Input	System Summary Output
`Node 1: 42.5°C, Node 2: 88.1°C, Node 1: 51.0°C`	`Total Events: 3 | Peak Global Temp: 88.1°C` `System Status: OPERATIONAL (Logged via static engine)`